

LN16. 分类与回归树 – KD 树和球树

何 琨

计算机学院数据挖掘与机器学习实验室

华中科技大学 Hopcroft 计算科学研究中心

邮箱: brooklet60@hust.edu.cn

2026 年 4 月



- 1 k-NN 回顾
 - k-NN 的时间复杂度
 - KD 树的设计动机
- 2 KD 树
 - KD 树的总体思路
 - 方法的有效性证明
 - KD 树的数据结构
 - KD 树的优缺点
- 3 Ball 树
 - Ball 树的概念
 - Ball 树的构造
 - Ball 树的使用
- 4 总结

- 1 k-NN 回顾
 - k-NN 的时间复杂度
 - KD 树的设计动机
- 2 KD 树
 - KD 树的总体思路
 - 方法的有效性证明
 - KD 树的数据结构
 - KD 树的优缺点
- 3 Ball 树
 - Ball 树的概念
 - Ball 树的构造
 - Ball 树的使用
- 4 总结

假设

- 设输入数据是在 d 维空间。
- 设已经处理了一些输入，想推断再增加一个数据点的时间复杂度。

回顾

- **训练时**，k-NN 简单地记住它看到的每个数据点的标签。
- 因此，增加一个数据点的时间复杂度是 $O(d)$ 。
- **测试时**，需要计算新增数据点与所有训练过的数据点之间的距离。设 n 为训练过的数据点数量，那么对新增点训练的时间复杂度是 $O(dn)$ 。
- **分类一个测试点**，时间复杂度也是 $O(dn)$ 。

回顾

- **训练时**, k -NN 简单地记住它看到的每个数据点的标签。
- 因此, 增加一个数据点的时间复杂度是 $O(d)$ 。
- **测试时**, 需要计算新增数据点与所有训练过的数据点之间的距离。设 n 为训练过的数据点数量, 那么对新增点训练的时间复杂度是 $O(dn)$ 。
- **分类一个测试点**, 时间复杂度也是 $O(dn)$ 。

设计动机

为了达到最好的准确性, 希望训练数据集尽量大 ($n \gg 0$)。
但这很快就会成为测试期间的一个瓶颈。

Q: 能否找到一种途径, 在测试时使 k -NN 更快?

A: 如果找到合适的数据结构, 则可以达成。

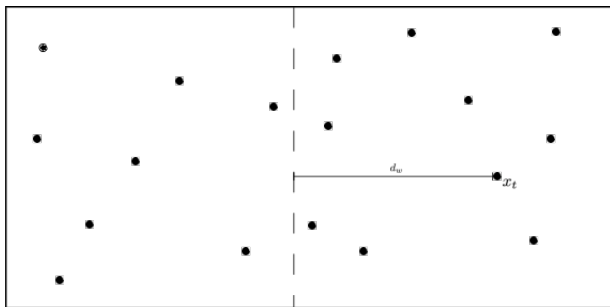
- 1 k-NN 回顾
 - k-NN 的时间复杂度
 - KD 树的设计动机
- 2 KD 树
 - KD 树的总体思路
 - 方法的有效性证明
 - KD 树的数据结构
 - KD 树的优缺点
- 3 Ball 树
 - Ball 树的概念
 - Ball 树的构造
 - Ball 树的使用
- 4 总结

KD 树的总体思路

KD 树 (K-Dimensional Trees) 的总体思想是对特征空间进行划分。
我们希望：能丢弃大量的数据点，因为其分布比最近 k 个邻居更远。

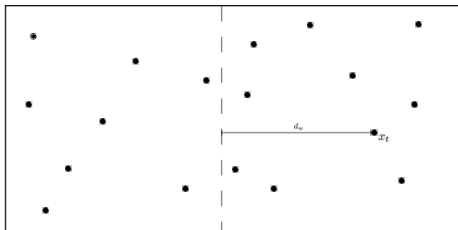
按以下方式对数据划分：

- ① 沿着某个特征把数据分成两半，如按左右划分。
- ② 对于每个训练输入，记住它所在的是哪一半。



划分特征空间

这种划分方式如何加速测试？



首先，考虑一个邻居的特例，即 1NN 算法。

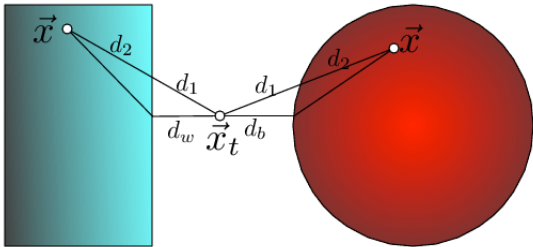
- 1 确定测试点位于哪一侧，如在右侧。
- 2 找到同一侧 x_t 的最近邻居 x_{NN}^R 。R 表示此最近邻也在右边。
- 3 计算 x_t 和分割“线”（超平面）之间的距离。将其标记为 d_w 。若 $d_w > d(x_t, x_{NN}^R)$ ，则可丢弃左边的所有点，从而得到 2 倍的加速。

即：如果到划分超平面的距离大于到最近邻居的距离，则可知该划分区内没有任何数据点可以更近。

从而可以避免计算该测试点到该分区中所有点的距离。

Proof

我们利用三角不等式，证明如上论述。



Proof. 设 $d(x_t, x)$ 表示测试点 x_t 和某候选点 x 之间的距离。若 x 在划分线的另一边, 则该距离可分解为两部分: $d(x_t, x) = d_1 + d_2$

其中, d_1 是在 x_t 所在“墙”一侧的距离 (上图右侧),

d_2 是在 x 所在“墙”一侧的距离（上图左侧）。

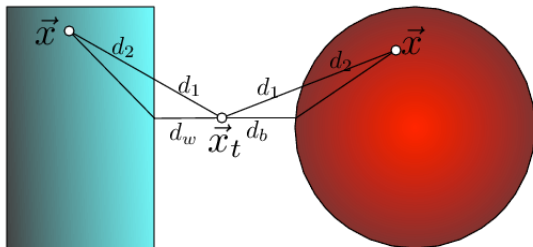
同样，用 d_w 表示从 x_t 到墙的最短距离。

因 $d_1 > d_w$, 可得:

$$d(x_t, x) = d_1 + d_2 \geq d_w + d_2 \geq d_w.$$

若 d_w 已经大于到当前最佳候选的最近邻居点的距离, 则可以安全地放弃 x 作为候选点。

三角形不等式的图解



用 KD 树和球树划分 $\vec{x}_t$ 和 $\vec{x}$ 之间距离的边界 (这里 $\vec{x}$ 画了两次, 左为 KD 树示例, 右为球树示例)。

距离可以分解为两个分量 $d(\vec{x}_t, \vec{x}) = d_1 + d_2$, 其中 d_1 是到 ball/box 分量, d_2 是 ball/box 里面的分量。

在这两种情况下, d_1 的下界分别是到墙的距离 d_w 或到 ball 的距离 d_b , 即:

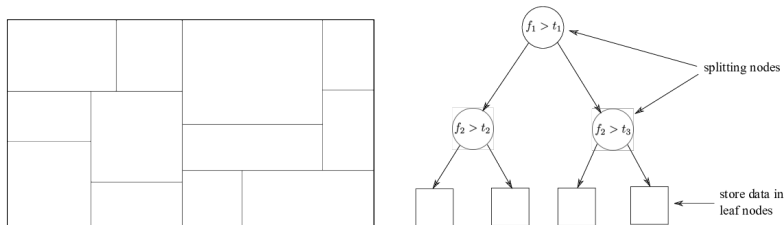
$$d(\vec{x}_t, \vec{x}) = d_1 + d_2 \geq d_w + d_2 \geq d_w$$

思考

构造一个该方法的反例（即不能通过划分删除另外一侧的点集）。

KD 树的结构

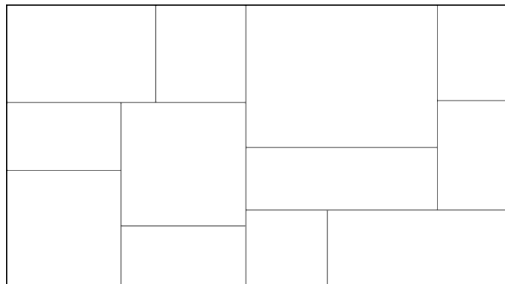
- KD 树是一棵二叉树，每个节点都有 2 个分支。
- 每个非叶子节点均可被视为隐式地生成一个分割超平面，将空间分为两个部分，称为半空间。
- 超平面左侧的点由该节点的左子树表示，超平面右侧的点由右子树表示。
- 超平面方向的选择方法如下：树中的每个节点都与 d 维中的一个维度相关联，超平面垂直于该维度的坐标轴。



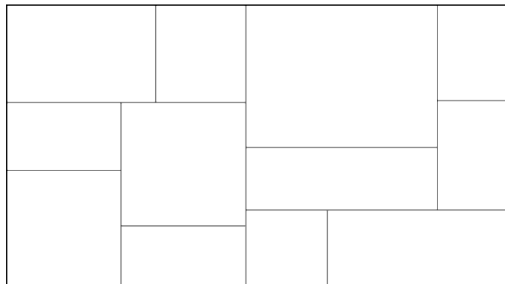
图：划分后的特征空间及其对应的 KD 树

树结构:

- ① 按某特征进行划分：递归地在某个特征上将数据划分成两半。
- ② 选择进行划分的特征：一个很好的启发式是选择方差最大的特征。



例子



问题:

- 哪些划分区可以被剪枝?
- 哪些划分区必须被搜索, 按照什么样的顺序来搜索?

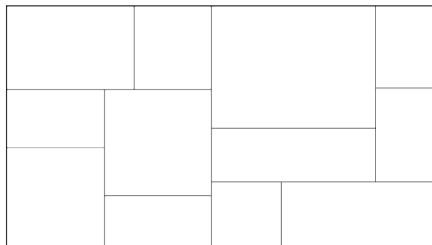
KD 树的优缺点

优点:

- 可精确计算
- 容易构造

缺点:

- 维度灾难使得 KD 树对于高维数据很低效
- 所有的分割都是与坐标轴平行的。



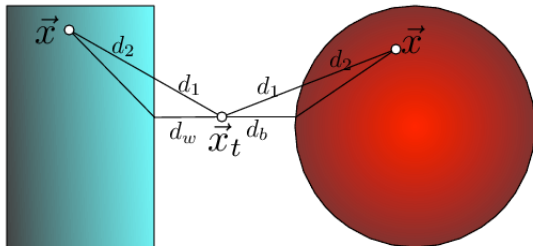
近似方法: 限制搜索到 m 层叶子即停机。

- 1 k-NN 回顾
 - k-NN 的时间复杂度
 - KD 树的设计动机
- 2 KD 树
 - KD 树的总体思路
 - 方法的有效性证明
 - KD 树的数据结构
 - KD 树的优缺点
- 3 Ball 树
 - Ball 树的概念
 - Ball 树的构造
 - Ball 树的使用
- 4 总结

Ball 树的概念

- 类似于 KD 树，但使用超球体 (ball) 代替矩形 (box)。
- 和前面的分析类似，可以用三角不等式来分析距离

$$d(x_t, x) = d_1 + d_2 \geq d_b + d_2 \geq d_b$$



- 如果到球的距离 d_b 大于到当前最近邻的距离，则可以安全地忽略球内的所有点。
- 球结构使得我们可以沿着数据点所在的低维流形来划分数据，而不用划分整个特征空间 (如 KD 树)。

Ball 树的构造

输入: 集合 S , $n = |S|$, 参数 k

Algorithm 1 Balltree in Pseudo-code

```
1: procedure BALLTREE( $S, k$ )  
2:   if  $|S| < k$  then stop end if ▷ Return leaf containing  $S$   
3:   pick  $x_0 \in S$  uniformly at random  
4:   pick  $x_1 = \operatorname{argmax}_{x \in S} d(x_0, x)$   
5:   pick  $x_2 = \operatorname{argmax}_{x \in S} d(x_1, x)$   
6:    $\forall i = 1 \dots |S|, z_i = (x_1 - x_2)^T x_i \quad \leftarrow$  project data onto  $(x_1 - x_2)$   
7:    $m = \operatorname{median}(z_1, \dots, z_{|S|})$   
8:    $S_L = \{x \in S: z_i < m\}$   
9:    $S_R = \{x \in S: z_i \geq m\}$   
10:  Return tree:  
    – center  $c = \operatorname{mean}(S)$   
    – radius  $r = \max_{x \in S} d(x, c)$   
    – children: Balltree( $S_L, k$ ) and Balltree( $S_R, k$ )  
11: end procedure
```

注意: 步骤 3 & 4 选择使 $(x_1 - x_2)$ 尽量大的方向。

与 KD 树类似:

在低维上比 KD 树慢 ($d \leq 3$), 但在高维上要快得多。

两者都受到维数灾难的影响,

如果数据具有局部结构 (如位于低维流形上), Ball 树往往仍然有效。

- 1 k-NN 回顾
 - k-NN 的时间复杂度
 - KD 树的设计动机
- 2 KD 树
 - KD 树的总体思路
 - 方法的有效性证明
 - KD 树的数据结构
 - KD 树的优缺点
- 3 Ball 树
 - Ball 树的概念
 - Ball 树的构造
 - Ball 树的使用
- 4 总结

- k -NN 在测试时很慢，因为它做了很多不必要的计算。
- KD 树对特征空间进行分区，从而可以排除与最近 k 邻居距离更远的整个分区。然而，分割面是平行于坐标轴的，不能很好地扩展到更高的维度。
- 球树划分数据点所在的流形，而不是整个空间。这使得它在高维空间中表现得更好。

The End